

USB-UART Text Console

Step-by-step Vivado + Vitis flow to type matrices into the 4×4 weight-stationary systolic-array multiplier and read results back in a serial terminal

Target part `xc7z020c1g484-1` · Vivado + Vitis 2019.1 or later · Design under test `accelerator_system_top` (unmodified) · Companion document: [VIO_ILA.pdf](#)

Contents

1. What this method gives you
2. The data path, and why the PS is involved
3. Prerequisites, board setup, cable drivers
4. The design under test — protocol and data layout
5. The AXI4-Lite wrapper `accele1_axi.sv`
6. Step 1 — create the Vivado project
7. Step 2 — add the sources
8. Step 3 — create the block design
9. Step 4 — add and configure the Zynq PS
10. Step 5 — add `accele1_axi` to the block design
11. Step 6 — connection automation
12. Step 7 — assign the address
13. Step 8 — validate, wrap, set top
14. Step 9 — generate the bitstream
15. Step 10 — export hardware (`.xsa`)
16. Step 11 — the Vitis platform project
17. Step 12 — the Vitis application project
18. Step 13 — BSP settings: route stdio to UART1
19. Step 14 — open the serial terminal
20. Step 15 — program the FPGA and run
21. Step 16 — use the console
22. Register map reference
23. How the software talks to the hardware
24. Troubleshooting
25. Extending to Method 4
26. Appendix A — data packing reference
27. Appendix B — build checklist

1. What this method gives you

Method 1 makes you enter thirty-two int8 bytes on eight slide switches, one button press at a time. Method 2 makes you type hex words into a Vivado dashboard. **Method 3 gives you a text console:** open a serial terminal, press `d` for the demo or `w` to type your own matrix, press `r` to run, press `p` to print. Still no external peripherals — the ZedBoard's USB-UART bridge is on the board itself.

```

accel> d
demo matrices loaded (expect result row 3 = 28 32 36 40).

accel> r
running...
done.

accel> p
C = A x W :
[  1   2   3   4 ]
[  5   6   7   8 ]
[  9  10  11  12 ]
[ 28  32  36  40 ]

accel>
```

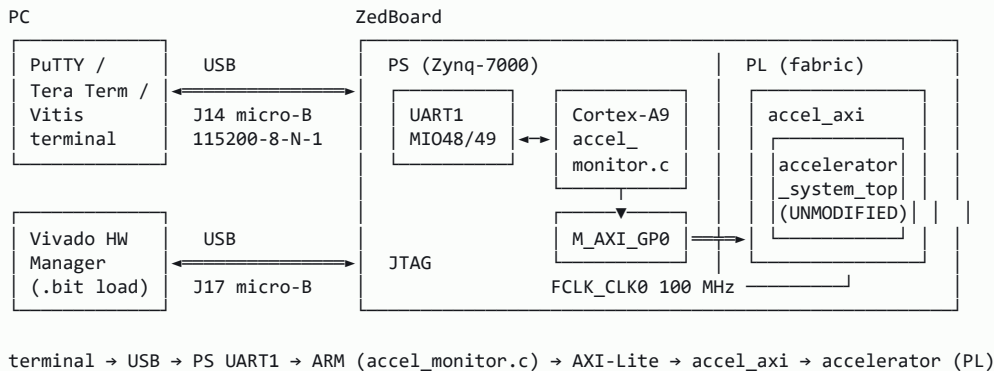
2. The data path, and why the PS is involved

THE ZEDBOARD USB-UART IS A PS RESOURCE, NOT A PL RESOURCE

On the ZedBoard the USB-to-serial bridge is hard-wired to the Zynq

PROCESSING SYSTEM

— UART1 on MIO48/49. Those pins are not routable from the PL fabric. So a "no external peripherals" text console cannot be a pure-PL design: it needs a minimal PS that owns the UART, with an AXI4-Lite bridge carrying commands from the ARM core into the accelerator. That bridge is `accel_axi.sv`.



TWO USB CABLES, TWO DIFFERENT JOBS

J17 (PROG)

carries JTAG — Vivado uses it to load the bitstream and Vitis uses it to download and debug the ARM application.

J14 (UART)

carries the serial console. You want

BOTH

plugged in for this method. Method 2 (VIO+ILA) uses only J17.

3. Prerequisites, board setup, cable drivers

3.1 Tools

- **Vivado and Vitis, the same version**, 2019.1 or later. A mismatched pair is a common source of "cannot open the `.xsa`" errors. In 2019.1 and later the tool is called Vitis; in 2018.3 and earlier it was SDK — the flow is equivalent.
- **Cable drivers** — accept Vivado's "Install Cable Drivers" prompt, or run the installer from `<Xilinx>/Vivado/<ver>/data/xicom/cable_drivers/`.
- **USB-UART bridge driver** — see 3.3.
- Optionally the **Digilent board files** for the ZedBoard. These are *strongly recommended* for this method, because the board preset configures the PS DDR, clocks and MIO peripherals correctly in one click. Without them you must configure the PS by hand.

3.2 Board setup

Item	Setting	Why
Power JP3 / SW8	USB or wall adapter; power ON	PGOOD (LD13) must be lit.
Boot mode JP7–JP11	All five to GND = JTAG	Vitis downloads the ELF over JTAG rather than the PS booting from QSPI/SD. Confirm against the ZedBoard Hardware User Guide for your board revision.
JTAG cable	Micro-USB into J17	Bitstream download + ARM debug.
UART cable	Micro-USB into J14	The serial console.

3.3 The USB-UART driver and COM port

Which bridge chip the board carries depends on its revision:

Board revision	Bridge chip	Driver
ZedBoard rev C and earlier	Cypress CY7C64225	Cypress USB-to-UART driver (bundled by Digilent)
ZedBoard rev D	Silicon Labs CP2104	Silicon Labs CP210x VCP driver

Both enumerate as a standard **USB Serial Port (COMx)**. After plugging J14 in and powering the board, open **Device Manager** → **Ports (COM & LPT)** and note the COM number. If it appears with a warning triangle, install the matching driver above.

NOTE THE COM NUMBER NOW

You need it in step 14. If several COM ports are present, unplug J14, refresh Device Manager, plug it back in, and see which one appears.

4. The design under test — protocol and data layout

The accelerator is used exactly as the simulation testbench used it. Nothing about the RTL changes.

4.1 Host protocol

1. hold rst_n low, then release
2. while (!busy): write weight rows via host_w_* (4 words)
write activations via host_a_* (num_vectors words)
3. pulse start, with num_vectors / act_base / out_base valid
4. wait for the done pulse (job length = num_vectors + 13 clocks)
5. while (!busy): read results via host_o_addr -> o_rdata

4.2 The four facts the software must respect

#	Fact	Software consequence
1	Job length = <code>num_vectors</code> + 13 clocks. Four vectors = 17 clocks = 170 ns . If the weight matrix has not changed since the last job the FSM skips WLOAD and it is <code>num_vectors</code> + 9 = 13 clocks = 130 ns .	Faster than a single AXI register write from the ARM. By the time the CPU's next instruction executes, the job is long finished. The polling loop in <code>run_job()</code> almost always exits on its first read. Software needs no change to get the reuse speed-up — just do not rewrite the weight registers between jobs.
1b	The FSM has 5 states, not 6. Encoding: 0 IDLE, 1 WLOAD, 2 COMPUTE, 3 DRAIN, 4 DONE . The old FLUSH phase (<code>ROWS+2</code> = 6 clocks between WLOAD and COMPUTE) has been removed.	Any software that decodes <code>STATUS[4:2]</code> to name the phase must use the new table. Nothing else changes: the driver only ever waits on sticky-done.
2	<code>done</code> is a one-cycle pulse (10 ns).	Software polling cannot possibly catch it. <code>accel_axi</code> latches it into a sticky-done bit, cleared automatically when a new <code>start</code> is written. Software polls <code>STATUS.bit1</code> .
3	Host writes are masked while <code>busy==1</code>.	Load everything <i>before</i> writing START. The monitor's <code>run_job()</code> does exactly this.
4	<code>output_mem</code> has one clock of read latency after <code>host_o_addr</code> changes.	10 ns versus an AXI read of tens of nanoseconds — comfortable, but <code>print_result()</code> inserts two dummy STATUS reads anyway as belt-and-braces.

4.3 Data layout — get a lane wrong and the matrix silently transposes

<code>weight_mem[r]</code>	<code>[31:24][23:16][15:8][7:0]</code>	<code>= W[r][3] W[r][2] W[r][1] W[r][0]</code>	(lane c = column c)
<code>a_mem[i]</code>	<code>[31:24][23:16][15:8][7:0]</code>	<code>= A[i][3] A[i][2] A[i][1] A[i][0]</code>	(lane r = row r)
<code>output_mem[i]</code>	<code>[127:96][95:64][63:32][31:0]</code>	<code>= C[i][3] C[i][2] C[i][1] C[i][0]</code>	(lane c, int32)

This is what `pack()` in `accel_monitor.c` implements:

```
static u32 pack(signed char l0, signed char l1, signed char l2, signed char l3){
    return (((u32)(u8)l0)          | (((u32)(u8)l1) << 8)
           | (((u32)(u8)l2) << 16) | (((u32)(u8)l3) << 24);
}
```

WRITE WEIGHT ROWS IN NATURAL ORDER 0, 1, 2, 3

The design's "weight rows load bottom-first" rule is the **AGU'S** job — it counts `w_addr` down 3, 2, 1, 0 during WLOAD. The software writes `weight_mem[r] = W` row *r* in natural order, which is what `load_weights()` does. Pre-reversing produces a wrong result that still looks plausible.

5. The AXI4-Lite wrapper `accel_axi.sv`

`accel_axi` is a minimal AXI4-Lite slave that exposes the accelerator's host ports as memory-mapped registers. It instantiates `accelerator_system_top` unmodified and adds three things:

1. **A standard AXI4-Lite register file.** Nine write registers, five read registers.
2. **One-cycle command pulses.** Writes to `W_COMMIT`, `A_COMMIT` and `START` produce single-clock strobes, because the accelerator's control inputs are pulses, not levels:

```
start_p <= 0; w_commit_p <= 0; a_commit_p <= 0;    // default low every cycle
if (slv_wr) case (widx)
  6'd4: begin w_addr_p <= S_AXI_WDATA[1:0]; w_commit_p <= 1'b1; end
  6'd6: begin a_addr_p <= S_AXI_WDATA[7:0]; a_commit_p <= 1'b1; end
  6'd7: start_p <= 1'b1;
endcase
```

3. **Sticky done.** Fact 2 above:

```
if      (!S_AXI_ARESETN) done_sticky <= 1'b0;
else if (start_p)       done_sticky <= 1'b0;    // arm for the new job
else if (done)          done_sticky <= 1'b1;    // latch the 10 ns pulse
```

Note that `rst_n` is tied to `S_AXI_ARESETN`, so the accelerator is reset by the AXI peripheral reset — no separate reset logic is needed.

6. 1 Create the Vivado project

1. Launch Vivado → **Create Project** → *Next*.
2. Project name e.g. `accel_uart`. Choose a path with **no spaces** and keep it short — Vitis workspaces inherit long paths and Windows path-length limits bite. *Next*.
3. **RTL Project**, tick *Do not specify sources at this time*. *Next*.
4. Select the target:
 - **Boards** tab → **ZedBoard Zynq Evaluation and Development Kit** — preferred, because it unlocks the one-click PS preset in step 4.
 - Or **Parts** tab → `xc7z020c1g484-1` if you do not have the board files.

Next → **Finish**.

7. 2 Add the sources

1. *Flow Navigator* → *Add Sources* → *Add or create design sources* → *Add Files*.
2. Select all **13** `.sv` files in `...SIMPLE_ACCEL_RTL/`:

```
MAC.sv          array.sv          delay_pipe.sv
input_buf.sv     output_skewbuf.sv  a_mem.sv
weight_mem.sv    outmem.sv         controller.sv
agu.sv           wrapper_array_buf.sv wrapper_mem_arr_buf.sv
wrapper_full_system.sv
```

3. Also add `FPGA_IMPL/rtl/accel_axi.sv`.
4. Tick **Copy sources into project** if you want a self-contained project. **Finish**.

SET THE FILE TYPE TO SYSTEMVERILOG

In the *Sources* pane, verify every file shows as

SYSTEMVERILOG

. If any shows as Verilog, right-click → *Set File Type* → *SystemVerilog*. Symptom of getting this wrong: hundreds of syntax errors on `logic`, `always_ff`, `always_comb`, or on the unpacked-array port `result [0:3]`.

The hierarchy should now show `accel_axi` at the top with `accelerator_system_top` beneath it, resolving cleanly down to the PEs. If any module shows as missing, a file was not added.

8. 3 Create the block design

1. *Flow Navigator* → *IP INTEGRATOR* → *Create Block Design*.
2. Design name: `system`. Directory: `<Local to Project>`. **OK**.
3. An empty BD canvas opens.

9. 4 Add and configure the Zynq PS

1. Click + (Add IP) on the canvas, type `zynq`, double-click **ZYNQ7 Processing System**.
2. A green banner appears: **Run Block Automation**. Click it.
3. In the dialog tick **Apply Board Preset** and leave *Cross Trigger In/Out* disabled. **OK**. The preset configures DDR3, the MIO peripheral assignment and the clock tree for the ZedBoard. If you selected the raw part instead of the board, this checkbox is absent — see the note below.

NO BOARD FILES? CONFIGURE THE PS MANUALLY

Double-click the PS block and set, at minimum: *DDR Configuration* → memory part `MT41K128M16 JT-125` (ZedBoard's DDR3); *MIO Configuration* → *I/O Peripherals* → enable

UART 1

on

MIO 48..49

; *Clock Configuration* →

FCLK_CLK0 = 100 MHZ

; *PS-PL Configuration* → enable

M_AXI_GP0

. This is why the board files are worth installing.

9.1 Verify three things in Re-customize IP

Double-click the ZYNQ7 block and check each of these — **all three are required**:

Page	Setting	Why
MIO Configuration → I/O Peripherals	UART 1 enabled, MIO 48..49	This is the on-board USB-UART. UART 0 is <i>not</i> wired to J14 — enabling the wrong one gives you a working build and a silent terminal.
Clock Configuration → PL Fabric Clocks	FCLK_CLK0 = 100 MHz , enabled	Clocks the whole PL side: the AXI interconnect, <code>accel_axi</code> , and the accelerator itself.
PS-PL Configuration → AXI Non Secure Enablement → GP Master AXI Interface	M AXI GP0 interface ticked	The port the ARM uses to reach <code>accel_axi</code> . Without it, Connection Automation has nothing to connect and the address editor stays empty.

Click **OK**.

10. 5 Add `accel_axi` to the block design

There are two routes. Try route A first; fall back to route B if Vivado does not recognise the AXI interface.

10.1 Route A — Add Module **QUICK**

1. Right-click empty canvas → **Add Module...**
2. Select `accel_axi` → **OK**.
3. **Verify the interface was inferred.** The block should show a single grouped pin labelled `S_AXI` with a + expander, plus separate `S_AXI_ACLK` and `S_AXI_ARESETN` pins. Vivado infers this from the standard `S_AXI_AWADDR / S_AXI_WVALID / ...` port naming.

If instead you see thirty-odd individual scalar pins, the inference failed — use route B.

10.2 Route B — package it as a custom IP **ROBUST**

1. *Tools* → *Create and Package New IP...* → *Next*.
2. Choose **Package your current project** → *Next*.
3. Pick an IP location outside the project (e.g. `.../ip_repo/accel_axi_1.0`) → *Next* → **Finish**. A packaging project opens.
4. On the **Ports and Interfaces** page, confirm `S_AXI` is recognised as an `aximm` interface with `S_AXI_ACLK` as its associated clock and `S_AXI_ARESETN` as its associated reset. If not, right-click → *Auto Infer Interface* → *AXI4-Lite Slave*, then set the clock/reset association.
5. On **Review and Package**, click **Package IP**.
6. Back in the main project: *Settings* → *IP* → *Repository* → + → add your `ip_repo` path → **OK**.
7. On the BD canvas, + (Add IP) → search `accel_axi` → double-click.

WHY ROUTE B IS MORE RELIABLE

Packaging makes the AXI interface, its clock/reset association, and the address-space declaration *explicit metadata* rather than something inferred from port names at BD-assembly time. It also gives you a reusable IP you can drop into other block designs. Route A is faster when it works.

11. 6 Connection automation

1. A green banner appears: **Run Connection Automation**. Click it.
2. In the dialog tick **All Automation** (the checkbox at the top of the tree) → **OK**.

Vivado now does all of the following automatically:

What it inserts / connects	Purpose
AXI Interconnect (or SmartConnect)	Bridges <code>M_AXI_GP0</code> to the <code>accel_axi/S_AXI</code> slave port.
Processor System Reset	Generates a synchronised, correctly sequenced <code>peripheral_aresetn</code> from <code>FCLK_RESET0_N</code> .
<code>FCLK_CLK0</code> → <code>S_AXI_ACLK</code> , interconnect clocks, <code>M_AXI_GP0_ACLK</code>	Everything on the PL side runs on one 100 MHz clock — no CDC anywhere in this design.
<code>peripheral_aresetn</code> → <code>S_AXI_ARESETN</code>	Which, inside <code>accel_axi</code> , is also the accelerator's <code>rst_n</code> .

Click the **Regenerate Layout** icon to tidy the canvas. You should see four blocks: ZYNQ7 PS → AXI Interconnect → `accel_axi`, with a Processor System Reset off to one side.

EVERYTHING IS SYNCHRONOUS TO FCLK_CLK0

The PS AXI master, the interconnect, `accel_axi`, and the accelerator all run on the same 100 MHz clock. There is no clock-domain crossing to reason about, which is one reason this design is a comfortable first PS/PL integration.

12. 7 Assign the address

1. *Window* → *Address Editor*.
2. Expand `processing_system7_0` → *Data*. You should see `accel_axi_0/S_AXI` with an **Offset Address** and a **Range**.
3. If it is unmapped, right-click → **Assign Address** (or click *Auto Assign Address*).
4. The usual result is `0x43C0_0000`, range **64K** (Vivado's minimum allocation is 4K; `accel_axi` only decodes 256 bytes of it, which is harmless).
5. **Write this address down**. It becomes `XPAR_ACCEL_AXI_0_S_AXI_BASEADDR` in `xparameters.h`, and `accel_monitor.c` uses it.

IF THE ADDRESS EDITOR IS EMPTY

Either `M_AXI_GP0` is not enabled in the PS (step 4, section 9.1), or the AXI interface on `accel_axi` was never inferred (step 5). Fix that before continuing — a build with an unmapped slave compiles fine and then hangs the ARM with a data-abort on the first register write.

13. 8 Validate, wrap, set top

1. Press **F6** (or *Validate Design*). Expect "Validation successful". Resolve any critical warnings before continuing — unconnected clock or reset pins are the usual culprits.
2. **Ctrl+S** to save the block design.
3. In the **Sources** pane, right-click `system.bd` → **Create HDL Wrapper** → choose **Let Vivado manage wrapper and auto-update** → **OK**.
4. Right-click the generated `system_wrapper` → **Set as Top** (it turns bold).

14. 9 Generate the bitstream

1. *Flow Navigator* → *Generate Bitstream*. Accept the prompt to run synthesis and implementation first. **OK**.
2. Wait. On a typical machine this takes 5–15 minutes.
3. When it completes, check *Reports* → *Timing* → *Report Timing Summary*: **WNS must be positive**. This design has ample margin at 100 MHz.

NO PL PIN CONSTRAINTS ARE NEEDED

Unlike Method 1 and Method 2, this design has

ZERO

PL-level top ports. The clock comes from the PS (`FCLK_CLK0`), and the UART pins are PS MIO, which the PS7 block configures directly. There is no XDC to write, and consequently no risk of a wrong `PACKAGE_PIN`.

15. 10 Export hardware (`.xsa`)

1. *File* → *Export* → *Export Hardware...*
2. Select **Include bitstream**. This is essential — without it, Vitis has no bitstream to program and the PL stays empty.
3. Choose an export path (default is the project directory) → **OK**. You now have `system_wrapper.xsa`.

16. 11 The Vitis platform project

1. Launch **Vitis** (*Tools* → *Launch Vitis IDE* from Vivado, or from the Start menu).
2. Choose a **workspace** directory — again, short path, no spaces.
3. *File* → *New* → *Platform Project...*
4. Name it e.g. `accel_platform` → *Next*.
5. On *Hardware Specification*, browse to the `system_wrapper.xsa` you just exported.
6. Set **Operating system** = `standalone`, **Processor** = `ps7_cortexa9_0`. Tick **Generate boot components** if offered. **Finish**.
7. Right-click the platform in the Explorer → **Build Project**. Wait for the BSP to compile.

17. 12 The Vitis application project

1. *File* → *New* → *Application Project...* → *Next*.
2. On *Platform*, select the `accel_platform` you just built → *Next*.
3. Application project name: `accel_monitor`. System project name defaults from it. *Next*.
4. Domain: `standalone_ps7_cortexa9_0` → *Next*.
5. Template: **Empty Application (C)** → **Finish**.
6. Copy `FPGA_IMPL/sw/accel_monitor.c` into the application's `src/` folder. Drag-and-drop into the Vitis Explorer works; choose *Copy files*.

CONFIRM THE BASE ADDRESS

Open <platform>/export/.../include/xparameters.h and search for ACCEL_AXI. You should find XPAR_ACCEL_AXI_0_S_AXI_BASEADDR 0x43C00000. accel_monitor.c picks this up automatically:

```
#ifdef XPAR_ACCEL_AXI_0_S_AXI_BASEADDR
    #define ACCEL_BASE    XPAR_ACCEL_AXI_0_S_AXI_BASEADDR
#else
    #define ACCEL_BASE    0x43C00000u    /* typical default; confirm in xparameters.h */
#endif
```

If the macro is named differently (Vivado derives it from the BD instance name), either rename your BD instance to accel_axi_0 or edit the #define.

18. 13 BSP settings: route stdio to UART1

THIS IS THE STEP MOST OFTEN SKIPPED, AND IT IS THE ONE THAT PRODUCES A SILENT TERMINAL

By default the standalone BSP may route `stdin / stdout` to `ps7_uart_0`. On the ZedBoard the USB bridge is on **UART1**

. Get this wrong and everything builds, the application runs, and nothing ever appears in your terminal.

1. In the Explorer, open the **platform** project → double-click `platform.spr`.
2. Navigate to `standalone` on `ps7_cortexa9_0` → **Board Support Package** → **Modify BSP Settings...**
3. Select `standalone` in the left tree.
4. Set `stdin` = `ps7_uart_1` and `stdout` = `ps7_uart_1`.
5. **OK**, then **rebuild the platform**, then rebuild the application.

Then build the application: right-click `accel_monitor` → **Build Project**. Expect zero errors.

19. 14 Open the serial terminal

Any of these work. Open it **before** you run the application so you do not miss the banner.

Terminal	How
Vitis built-in	<i>Window</i> → <i>Show View</i> → <i>Vitis Serial Terminal</i> → click + → select the COM port → set the settings below → OK . Most convenient, since it lives in the same window as the Run button.
PuTTY	Connection type Serial , Serial line COMx, Speed 115200. Under <i>Connection</i> → <i>Serial</i> , set Flow control to None . Open.
Tera Term	<i>File</i> → <i>New connection</i> → <i>Serial</i> → pick COMx. Then <i>Setup</i> → <i>Serial port</i> → 115200, 8 bit, none, 1 stop, none.

Setting	Value	Note
Baud rate	115200	The BSP default for the ZedBoard preset. Wrong baud = garbage characters.
Data bits	8	Collectively "8-N-1".
Parity	None	
Stop bits	1	
Flow control	None	The board has no RTS/CTS wiring. Leaving this on hardware flow control makes the terminal appear dead.

20. 15 Program the FPGA and run

1. Board powered, **both** USB cables connected (J17 and J14), terminal open.
2. In Vitis: *Xilinx* → *Program Device* (or right-click the system project → *Program FPGA*). Confirm the bitstream path is the one from your `.xsa`. Click **Program**. The blue DONE LED (LD12) lights when the PL is configured.
3. Right-click the `accel_monitor` application → **Run As** → **Launch Hardware (Single Application Debug)**. Vitis resets the PS, downloads the ELF over JTAG and starts it.
4. The help banner appears in the terminal:

```

4x4 systolic accelerator monitor
d  load built-in demo matrices
w  enter weight matrix W[r][c]
a  enter activation matrix A[i][r]
r  run the job
p  print the result matrix
?  this help

```

```
accel>
```

21. 16 Use the console

21.1 The built-in demo

Press **d**, then **r**, then **p**. This is the T5 golden case — the same vectors the simulation regression checks, so a mismatch here is unambiguously a hardware or setup problem.

A (4x4)		W (4x4)		C = A x W (4x4)
[1 0 0 0]		[1 2 3 4]		[1 2 3 4]
[0 1 0 0]	x	[5 6 7 8]	=	[5 6 7 8]
[0 0 1 0]		[9 10 11 12]		[9 10 11 12]
[1 1 1 1]		[13 14 15 16]		[28 32 36 40]

```

accel> d
demo matrices loaded (expect result row 3 = 28 32 36 40).

```

```

accel> r
running...
done.

```

```

accel> p
C = A x W :
[    1    2    3    4 ]
[    5    6    7    8 ]
[    9   10   11   12 ]
[   28   32   36   40 ]

```

Rows 0–2 reproduce the weight rows (activation rows 0–2 are one-hot selectors); the all-ones activation row 3 yields the column sums. One weight load, four back-to-back compute cycles — the weight-stationary economics of the design, visible in a terminal.

21.2 Your own matrices

```

accel> w
enter 16 values for weights, row-major W[r][c] (signed -128..127):
2 -1 3 0 1 4 -2 5 -3 2 1 1 0 1 2 -4
captured.

accel> a
enter 16 values for activations, row-major A[i][r] (signed -128..127):
1 2 3 4 5 -1 0 2 -2 3 1 1 0 0 0 0
captured.

accel> r
running...
done.

accel> p
C = A x W :
[   -5   17   10   -3 ]
[    9   -7   21  -13 ]
[   -4   17   -9   12 ]
[    0    0    0    0 ]

```

These are the architecture document's reference matrices. Values may be separated by spaces or newlines; `read_int()` skips whitespace and accepts a leading minus sign. Enter all 16 for each matrix — the parser does not prompt per element.

21.3 The command set

Key	Action	Detail
d	Load the built-in demo	Fills <code>W</code> and <code>A</code> in software only — nothing is sent to hardware until <code>r</code> .
w	Enter the weight matrix	16 signed values, row-major <code>W[r][c]</code> .
a	Enter the activation matrix	16 signed values, row-major <code>A[i][r]</code> .
r	Run the job	Writes weights, then activations, then <code>num_vectors=4</code> , <code>act_base=0</code> , <code>out_base=0</code> , then <code>START</code> ; polls sticky-done.
p	Print the result	Reads <code>output_mem[0..3]</code> and prints four signed int32 lanes per row.
?	Help	Reprints the banner.

22. Register map reference

All registers are 32 bits; the byte address is `index × 4` from `ACCEL_BASE` (typically `0x43C0_0000`).

Idx	Offset	Name	Access	Meaning
0	0x00	NUMVEC	W	<code>num_vectors[7:0]</code> — how many activation vectors this job streams
1	0x04	ACT_BASE	W	<code>act_base[7:0]</code> — first <code>a_mem</code> address of the job
2	0x08	OUT_BASE	W	<code>out_base[7:0]</code> — first <code>output_mem</code> address
3	0x0C	W_WDATA	W	32-bit weight word — four int8 lanes, lane <i>c</i> = column <i>c</i>
4	0x10	W_COMMIT	W	Write <code>w_addr[1:0]</code> → pulses <code>host_w_we</code> for one clock
5	0x14	A_WDATA	W	32-bit activation word — four int8 lanes, lane <i>r</i> = row <i>r</i>
6	0x18	A_COMMIT	W	Write <code>a_addr[7:0]</code> → pulses <code>host_a_we</code> for one clock
7	0x1C	START	W	Write anything → pulses <code>start</code> , and clears sticky-done
8	0x20	O_ADDR	W	<code>output_mem</code> read address for the read-back path
9	0x24	STATUS	R	bit0 = <code>busy</code> , bit1 = sticky <code>done</code> , bits[4:2] = <code>phase</code>
10	0x28	ORD0	R	<code>o_rdata[31:0]</code> — column 0 result (int32)
11	0x2C	ORD1	R	<code>o_rdata[63:32]</code> — column 1
12	0x30	ORD2	R	<code>o_rdata[95:64]</code> — column 2
13	0x34	ORD3	R	<code>o_rdata[127:96]</code> — column 3

22.1 STATUS bit layout

```
bit   31 ..... 5 | 4 3 2 | 1 | 0
      reserved (0) | phase | done_ | busy
                  | [2:0] | sticky |

phase encoding:  0 IDLE   1 WLOAD  2 COMPUTE  3 DRAIN  4 DONE
```

WHY DONE IS STICKY AND BUSY IS NOT

`busy` is a level that lasts `num_vectors + 18` clocks — still only 220 ns, so software essentially never observes it as 1. `done` is a *one-clock pulse*, which polling could never catch even in principle. Latching it in hardware is the only workable answer, and clearing it on the next `START` write means software never has to clear it explicitly.

23. How the software talks to the hardware

23.1 The register accessors

```
#define WR(idx,val)  Xil_Out32(ACCEL_BASE + ((idx)*4), (u32)(val))
#define RD(idx)      Xil_In32 (ACCEL_BASE + ((idx)*4))
```

23.2 Loading a matrix

Every write is a two-step: stage the data word, then commit it to an address. The commit write is what generates the one-clock `host*_we` strobe.

```
static void load_weights(void){
    for (int r = 0; r < 4; r++){
        WR(R_WDATA, pack(W[r][0], W[r][1], W[r][2], W[r][3]));
        WR(R_WCOMMIT, r);          /* commits weight_mem[r] */
    }
}
```

Note the **natural** row order 0,1,2,3 — see the box in section 4.3.

23.3 Running and waiting

```
static void run_job(void){
    load_weights();
    load_activations();
    WR(R_NUMVEC, 4);
    WR(R_ACTBASE, 0);
    WR(R_OUTBASE, 0);
    WR(R_START, 1);          /* pulse start (clears sticky done) */
    u32 guard = 0;
    while (!(RD(R_STATUS) & STATUS_DONE)) {
        if (++guard > 1000000u) { printf(" [timeout waiting for done]\n"); return; }
    }
}
```

The order matters: everything is loaded *before* `START`, because port arbitration masks host writes while `busy==1`. The job then takes 17 clocks (170 ns) — shorter than a single AXI read — so the polling loop virtually always exits on its first iteration. The guard counter turns a hardware hang into a printed message instead of a frozen console.

23.4 Reading results

```
static void print_result(void){
    printf(" C = A x W :\n");
    for (int row = 0; row < 4; row++){
        WR(R_OADDR, row);          /* select output_mem[row] */
        (void)RD(R_STATUS);         /* small delay: let read latency settle */
        (void)RD(R_STATUS);
        int c0 = (int)RD(R_ORD0 + 0);
        ...
    }
}
```

The two dummy STATUS reads cover `output_mem`'s one clock of read latency. An AXI read already takes far longer than 10 ns, so they are belt-and-braces rather than strictly necessary — but they cost nothing and remove a whole class of subtle failure.

24. Troubleshooting

24.1 Nothing appears in the terminal

Check	Fix
<code>stdin / stdout</code> not set to <code>ps7_uart_1</code>	The most common cause. Section 18. Rebuild the platform <i>and</i> the application afterwards.
UART1 not enabled in the PS	Re-customize ZYNQ7 → MIO Configuration → I/O Peripherals → tick UART 1 , MIO 48.49. Regenerate the bitstream.
Terminal on the wrong COM port	Device Manager → Ports. Unplug/replug J14 to identify it.
J14 not connected	J17 is JTAG; J14 is the UART. You need both.
Flow control set to hardware	Set it to None . The board has no RTS/CTS wiring.
Application never launched	<i>Run As</i> → <i>Launch Hardware</i> . Programming the FPGA alone does not start the ARM.

24.2 Garbage characters

Check	Fix
Baud rate mismatch	Set the terminal to 115200 . Confirm the BSP's UART baud rate matches.
Wrong data format	8 data bits, no parity, 1 stop bit.
Two terminals on the same port	Close one. A COM port can have a single owner.

24.3 Console works but results are wrong or zero

Symptom	Cause and fix
All results read 0	The bitstream was never loaded (only the ELF was), so the PL is empty and reads return 0. Run <i>Program FPGA</i> first. Also confirm you exported the <code>.xsa</code> with Include bitstream ticked.
Result matrix is transposed or scrambled	Lane order in the packed word. Lane 0 is the low byte. <code>weight_mem[r]</code> lane <code>c = W[r][c]</code> ; <code>a_mem[i]</code> lane <code>r = A[i][r]</code> . See Appendix A.
Row 3 is right (<code>28 32 36 40</code>) but rows 0–2 are wrong	Weight rows written in reverse. Write <code>weight_mem[0..3]</code> in natural order — bottom-first is the AGU's job. This failure hides in row 3 because the all-ones activation row is a column sum and is insensitive to row permutation.
Results are stale — the previous run's values	Confirm <code>START</code> is actually written (register index 7) and that sticky-done goes to 1. If <code>run_job()</code> printed a timeout, the job never completed.
<code>[timeout waiting for done]</code>	Either the PL is unprogrammed (reads return 0, so bit1 never sets), or the address decode is wrong so writes go nowhere, or <code>num_vectors</code> was left at 0 in a modified version. Check <code>ACCEL_BASE</code> against <code>xparameters.h</code> .

24.4 Build and integration problems

Symptom	Cause and fix
Address Editor is empty	<code>M_AXI_GP0</code> not enabled in the PS, or the AXI interface was never inferred on <code>accel_axi</code> . Sections 9.1 and 10.
<code>accel_axi</code> shows 30 loose pins instead of a grouped <code>S_AXI</code>	Interface inference failed. Use route B (package as IP) in section 10.2.
Hundreds of syntax errors on <code>logic</code> / <code>always_ff</code>	Files added as Verilog. Right-click each → <i>Set File Type</i> → <i>SystemVerilog</i> .
Vitis cannot open the <code>.xsa</code>	Vivado and Vitis versions differ. Use a matched pair.
<code>XPAR_ACCEL_AXI_0_S_AXI_BASEADDR</code> undefined	Vivado derives the macro from the BD instance name. Search <code>xparameters.h</code> for the actual name, or rename the BD instance to <code>accel_axi_0</code> and rebuild.
Data abort / prefetch abort on the first register write	The slave is unmapped or <code>ACCEL_BASE</code> is wrong. Verify both the Address Editor entry and <code>xparameters.h</code> .
Validate Design reports unconnected clock or reset	Re-run <i>Connection Automation</i> with All Automation ticked; check the Processor System Reset block was inserted.
Device not found under Program FPGA	Cable drivers, or J17 not connected, or boot-mode jumpers not in JTAG. Section 3.2.

WHEN THE CONSOLE IS HEALTHY BUT YOU CANNOT TELL WHY THE NUMBERS ARE WRONG

Add an ILA to this same block design and watch the run cycle-by-cycle — the companion `VIO_ILA.pdf` covers the setup, the expected phase-length map (4 / N / 8 / 1) and the result checkpoints. The two methods compose: keep the UART for comfortable data entry and use the ILA for visibility.

25. Extending to Method 4

Method 3 stands up the PS + AXI base. Method 4 is what you add for real workloads — still with no external peripherals.

Extension	How
Longer jobs from the console	The cheapest win: <code>a_mem</code> holds 256 activation vectors against a single weight load. Extend <code>load_activations()</code> to loop over more rows and write <code>NUMVEC</code> accordingly. Once the pipe is full it is one vector per clock — that amortisation is the entire economic point of a weight-stationary array.
Stream with AXI-DMA	Add an AXI-DMA (or an AXI-Stream port on <code>accel_axi</code>) so the PS DMA's a block of activation vectors into <code>a_mem</code> and reads the result block back, instead of one register write per word.
Interrupt on done	Route the accelerator's <code>done</code> to a PL→PS interrupt (<code>IRQ_F2P</code>) and enable it in the PS7 GIC. The monitor becomes event-driven instead of polling the sticky-done bit. Keep the sticky bit anyway — it is still the only way to read completion status after the fact.
Run under Linux	With PetaLinux, expose <code>accel_axi</code> as a UIO device, <code>mmap</code> the register block from a user-space C or Python program, and print over the same console — now <code>ttyPS0</code> .
Benchmark	Feed up to 256 vectors and time the job with the PS global timer. Expected: <code>num_vectors + 13</code> clocks, i.e. 2.75 μ s for a full 256-vector job at 100 MHz.
Bare-metal boot from SD	Package the bitstream + FSBL + ELF into a <code>BOOT.BIN</code> so the board runs the console standalone at power-up, with no PC attached at all.

Appendix A — data packing reference

A.1 The three word formats

weight_mem[r]	bits	[31:24] [23:16] [15:8] [7:0]	
		W[r][3] W[r][2] W[r][1] W[r][0]	int8, lane c = COLUMN c
a_mem[i]	bits	[31:24] [23:16] [15:8] [7:0]	
		A[i][3] A[i][2] A[i][1] A[i][0]	int8, lane r = ROW r
output_mem[i]	bits	[127:96] [95:64] [63:32] [31:0]	
		C[i][3] C[i][2] C[i][1] C[i][0]	int32, lane c = COLUMN c

A.2 The demo matrices as raw register writes

If you ever bypass the menu and poke registers directly (from XSDB, or from your own application), this is the exact sequence for the T5 golden case:

```
WR(3, 0x04030201); WR(4, 0);      /* weight_mem[0] = W row0 = 1 2 3 4      */
WR(3, 0x08070605); WR(4, 1);      /* weight_mem[1] = W row1 = 5 6 7 8      */
WR(3, 0x0C0B0A09); WR(4, 2);      /* weight_mem[2] = W row2 = 9 10 11 12   */
WR(3, 0x100F0E0D); WR(4, 3);      /* weight_mem[3] = W row3 = 13 14 15 16  */

WR(5, 0x00000001); WR(6, 0);      /* a_mem[0] = A row0 = 1 0 0 0          */
WR(5, 0x00000100); WR(6, 1);      /* a_mem[1] = A row1 = 0 1 0 0          */
WR(5, 0x00010000); WR(6, 2);      /* a_mem[2] = A row2 = 0 0 1 0          */
WR(5, 0x01010101); WR(6, 3);      /* a_mem[3] = A row3 = 1 1 1 1          */

WR(0, 4); WR(1, 0); WR(2, 0);     /* num_vectors=4, act_base=0, out_base=0 */
WR(7, 1);                          /* START -- 17 clocks = 170 ns          */
while (!(RD(9) & 0x2)) ;           /* poll sticky done                     */

WR(8, 3); RD(9); RD(9);            /* select output_mem[3], settle          */
RD(10); RD(11); RD(12); RD(13);    /* -> 28, 32, 36, 40                    */
```

A.3 Packing an arbitrary matrix by hand

For a signed byte v , the hex byte is v for 0...127 and $v+256$ for -128...-1.

```
word = (byte3 << 24) | (byte2 << 16) | (byte1 << 8) | byte0

W = [  2  -1   3   0 ]      lane order in the word: [b3][b2][b1][b0]
    [  1   4  -2   5 ]      row 0 -> 00 03 FF 02 -> weight_mem[0] = 0x0003FF02
    [-3   2   1   1 ]      row 1 -> 05 FE 04 01 -> weight_mem[1] = 0x05FE0401
    [  0   1   2  -4 ]      row 2 -> 01 01 02 FD -> weight_mem[2] = 0x010102FD
                                row 3 -> FC 02 01 00 -> weight_mem[3] = 0xFC020100

A = [  1   2   3   4 ]      A0 -> 04 03 02 01 -> a_mem[0] = 0x04030201
    [  5  -1   0   2 ]      A1 -> 02 00 FF 05 -> a_mem[1] = 0x0200FF05
    [-2   3   1   1 ]      A2 -> 01 01 03 FE -> a_mem[2] = 0x010103FE

expected C = [-5  17  10  -3 ]  ORD3..ORD0 = FFFFFFFD 0000000A 00000011 FFFFFFFB
              [  9  -7  21 -13 ]  ORD3..ORD0 = FFFFFFF3 00000015 FFFFFFF9 00000009
              [-4  17  -9  12 ]  ORD3..ORD0 = 0000000C FFFFFFF7 00000011 FFFFFFFC
```

Negative int32 results appear as `FFFF_FFxx`. `print_result()` casts to `int`, so the console shows them correctly signed.

Appendix B — build checklist

✓	Step	Verification that it worked
☐	1 · Project created	Part reads <code>xc7z020c1g484-1</code> in the project summary
☐	2 · 13 RTL + <code>accel_axi.sv</code> added	Hierarchy resolves with no missing modules; all files typed SystemVerilog
☐	3 · Block design created	<code>system.bd</code> in Sources
☐	4 · ZYNQ7 PS + board preset	UART1 on MIO48/49 · FCLK_CLK0 = 100 MHz · M_AXI_GP0 enabled
☐	5 · <code>accel_axi</code> added	Block shows a grouped <code>S_AXI</code> pin, not 30 loose ones
☐	6 · Connection Automation	AXI Interconnect + Processor System Reset present; no unconnected pins
☐	7 · Address assigned	Address Editor shows <code>accel_axi_0/S_AXI</code> at <code>0x43C0_0000</code>
☐	8 · Validate + wrapper + top	"Validation successful"; <code>system_wrapper</code> is bold in Sources
☐	9 · Bitstream generated	Positive WNS in the timing summary
☐	10 · Hardware exported	<code>.xsa</code> exists and Include bitstream was ticked
☐	11 · Vitis platform built	BSP compiles with no errors
☐	12 · Application created	<code>accel_monitor.c</code> is in <code>src/</code>
☐	13 · BSP stdio → <code>ps7_uart_1</code>	Platform and application rebuilt afterwards
☐	14 · Terminal open	COMx at 115200-8-N-1, flow control None
☐	15 · Program FPGA + Run	DONE LED (LD12) lit; help banner in the terminal
☐	16 · <code>d · r · p</code>	Result row 3 = <code>28 32 36 40</code>

Design reference: `context_mkdown/` — chunk 06 (memory word and lane layout), chunk 07 (controller FSM and phase durations), chunk 11 (L3 wrapper, port arbitration, host protocol), chunk 12 (dataflow and timing derivations), chunk 13 (worked example / T5 golden case).

Sources used: `FPGA_IMPL/rtl/accel_axi.sv`, `FPGA_IMPL/sw/accel_monitor.c`, the 13 unmodified files in `...SIMPLE_ACCEL_RTL/`.

Functional baseline: 11/11 testbenches, 3 680 checks, 0 mismatches (`verification_recheck/`).

Companion document: `VIO_ILA.pdf` — Method 2, driving and observing the accelerator over JTAG.